

Phase 3: JavaScript機能実装のヒント集



JavaScript基本概念

ヒント1: 要素の取得方法

```
// HTMLの要素を取得
const element = document.getElementById('要素のID');
const element = document.querySelector('.クラス名');
const element = document.querySelector('#ID名');
```

ヒント2: イベントリスナーの設定

```
// ボタンがクリックされた時の処理
button.addEventListener('click', function() {
    // ここに実行したい処理を書く
});

// Enterキーが押された時の処理
input.addEventListener('keypress', function(event) {
    if (event.key === 'Enter') {
        // ここに実行したい処理を書く
    }
});
```

ヒント3: HTML要素の作成と追加

```
// 新しい要素を作成
const newElement = document.createElement('li');

// テキストを設定
newElement.textContent = '新しいタスク';

// 親要素に追加
parentElement.appendChild(newElement);
```



タスク管理アプリの実装ポイント

ヒント4: タスクの追加機能

```
function addTask() {  
  // 1. 入力欄の値を取得  
  const taskText = inputElement.value;  
  
  // 2. 空の場合は何もしない  
  if (taskText.trim() === '') {  
    return;  
  }  
  
  // 3. 新しいタスクを作成  
  // 4. リストに追加  
  // 5. 入力欄をクリア  
}
```

ヒント5: タスクの削除機能

```
// 削除ボタンを作成  
const deleteButton = document.createElement('button');  
deleteButton.textContent = '削除';  
deleteButton.addEventListener('click', function() {  
  // このタスクを削除  
  taskItem.remove();  
});
```

ヒント6: 入力値の取得とクリア

```
// 入力欄の値を取得  
const value = inputElement.value;  
  
// 入力欄をクリア  
inputElement.value = '';  
  
// 空白を除去して確認  
const trimmedValue = value.trim();  
if (trimmedValue === '') {  
  // 空の場合の処理  
}
```



よくある間違いとその解決法

間違い1: 要素が取得できない

✖ 問題例:

```
const button = document.getElementById('add-btn');  
// buttonがnullになってしまう
```

原因と解決法:

- HTMLのIDが間違っている → HTMLを確認
- JavaScriptがHTMLより先に読み込まれている → DOMContentLoadedを使用

✅ 正しい例:

```
document.addEventListener('DOMContentLoaded', function() {  
    const button = document.getElementById('add-btn');  
    // ここで要素を使用  
});
```

間違い2: イベントが発生しない

❌ 問題例:

```
button.onclick = addTask(); // 関数を実行している
```

✅ 正しい例:

```
button.addEventListener('click', addTask); // 関数を渡している  
// または  
button.addEventListener('click', function() {  
    addTask();  
});
```

間違い3: 入力値が取得できない

❌ 問題例:

```
const taskText = input.innerHTML; // innerHTMLは不適切
```

✅ 正しい例:

```
const taskText = input.value; // inputの値はvalueプロパティ
```



段階別チェックポイント

Step 1: 基本的な要素取得ができていますか？

- ☐ 入力欄の要素が取得できている
- ☐ 追加ボタンの要素が取得できている
- ☐ タスクリストの要素が取得できている

Step 2: イベントリスナーが設定されているか？

- ☐ ボタンクリックで関数が呼ばれる
- ☐ Enterキーで関数が呼ばれる

- ☐ エラーが発生していない

Step 3: タスクの追加機能ができているか？

- ☐ 入力欄の値が取得できる
- ☐ 新しいli要素が作成される
- ☐ リストに追加される
- ☐ 入力欄がクリアされる

Step 4: タスクの削除機能ができているか？

- ☐ 各タスクに削除ボタンがある
- ☐ 削除ボタンで該当タスクが削除される



デバッグのコツ

コンソールを活用しよう

```
// 変数の値を確認
console.log('taskText:', taskText);

// 要素が取得できているか確認
console.log('button:', button);

// 関数が呼ばれているか確認
console.log('addTask関数が呼ばれました');
```

よく確認すべきポイント

1. 要素のIDやクラス名: HTMLとJavaScriptで一致しているか
2. タイプミス: 変数名や関数名に間違いがないか
3. ブラウザのコンソール: エラーメッセージが出ていないか



完成目標

- タスクの追加ができる
- タスクの削除ができる
- エラーが発生しない
- 完成版と同じ動作をする

動作確認方法:

1. テキストを入力して「追加」ボタンを押す
2. Enterキーでも追加できる

3. 各タスクの削除ボタンで削除できる
4. 空の入力では何も起こらない